

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

ОСВІТНІЙ КОМПОНЕНТ
«АЛГОРИТМИ І СТРУКТУРИ ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 5

Виконав:

студент групи кн-25-1

Рафієв Б. С.

Перевірив:

доцент кафедри КІЕ

Сидоренко В.М.

Кременчук 2026

Тема. Графи. Ациклічні графи

Мета: набути практичних навичок розв'язання задач топографічного сортування та оцінювання їх асимптотичної складності.

ХІД РОБОТИ

Завдання для самостійного розв'язання: Моделювання та топологічне сортування графа за допомогою алгоритму Кана

1. Умова задачі: Задано орієнтований ациклічний граф (ОАГ), який складається з наступних елементів:

- Множина вершин: $V = \{1, 2, 3, 4, 5\}$
- Множина орієнтованих ребер: $E = \{(1,2), (1,3), (2,4), (3,5), (4,5)\}$

Необхідно виконати топологічне сортування заданого графа, використовуючи покроковий аналіз вхідних степенів вершин.

Крок 1. Проводиться початковий аналіз графа та обчислюються вхідні степені для кожної з вершин (кількість ребер, що входять у вершину). Знайдемо вершини, які взагалі не мають вхідних ребер. На початку такою є лише вершина 1.

- Множина вершин без вхідних ребер: $S = \{1\}$
- Початковий список результату сортування: $L = []$

Крок 2. Вибираємо з множини S вершину 1 і переносимо її до підсумкового списку сортування. Після цього умовно видаляємо з графа саму вершину 1 та всі ребра, що з неї виходили, а саме: (1,2) та (1,3). Внаслідок цього вершини 2 та 3 втрачають свої єдині вхідні ребра і тепер їхній вхідний ступінь стає рівним нулю.

- Поточний список результату: $L = [1]$
- Оновлена множина вершин без вхідних ребер: $S = \{2, 3\}$

Крок 3. Вибираємо з множини S наступну вершину 2 і додаємо її до результату. Видаляємо з графа вихідне ребро (2,4). Оскільки для вершини 4 це ребро було єдиним вхідним, її вхідний ступінь зменшується до нуля, і вона стає доступною для подальшої обробки.

- Поточний список результату: $L = [1, 2]$

- Оновлена множина вершин без вхідних ребер: $S = \{3, 4\}$

Крок 4. Вибираємо з множини S вершину 3 та переносимо її до списку результату. Видаляємо вихідне ребро (3,5). Проте вершина 5 на даному етапі ще не може бути додана до множини S , оскільки в неї продовжує входити інше ребро – (4,5), тобто її вхідний ступінь все ще більший за нуль.

- Поточний список результату: $L = [1, 2, 3]$
- Оновлена множина вершин без вхідних ребер: $S = \{4\}$

Крок 5. Вибираємо з множини S вершину 4 і додаємо її до результату сортування. Видаляємо з графа ребро (4,5), яке веде до вершини 5. Тепер у вершину 5 більше не входить жодне ребро, її вхідний ступінь стає рівним нулю, і ми додаємо її до множини S .

- Поточний список результату: $L = [1, 2, 3, 4]$
- Оновлена множина вершин без вхідних ребер: $S = \{5\}$

Крок 6. Вибираємо останню вершину 5 з множини S і заносимо її до підсумкового списку. Оскільки вершина 5 не має жодних вихідних ребер, структура графа більше не змінюється.

- Поточний список результату: $L = [1, 2, 3, 4, 5]$
- Множина вершин без вхідних ребер S стає порожньою.

Остаточний результат топологічного сортування графа: $[1, 2, 3, 4, 5]$ або $[1, 3, 2, 4, 5]$ (обидва варіанти є правильними, оскільки вершини 2 та 3 є незалежними одна від одної).

2. Оцінка асимптотичної складності алгоритму:

- **Часова складність:** Становить $O(|V| + |E|)$, де $|V|$ – загальна кількість вершин у графі (у нашому випадку 5), а $|E|$ – кількість орієнтованих ребер (у нашому випадку 5). Алгоритм працює лінійно, оскільки кожна вершина додається та вилючається з черги рівно один раз, а кожне ребро переглядається і видаляється лише один раз під час зменшення ступенів сусідніх вузлів.

- **Просторова складність:** Становить $O(|V|)$ для збереження масиву або списку вхідних ступенів кожної вершини, а також для тимчасового збереження елементів у множині/черзі вершин без вхідних ребер та формування фінального послідовного списку.

Висновки: Під час виконання цієї практичної роботи я набув практичних навичок розв'язання задач топографічного сортування орієнтованого ациклічного графа за допомогою покрокового алгоритму Кана. Було детально проаналізовано зміну вхідних ступенів вершин та механізм усунення ребер залежностей. Також проведено теоретичне порівняння часових та просторових характеристик алгоритму Кана з алгоритмом пошуку в глибину (DFS), що дозволяє ефективно обирати метод сортування під конкретні структури даних та типи графів у майбутніх інженерних задачах.

Відповіді на контрольні запитання

1. Переваги та недоліки алгоритмів Кана та DFS: Головна перевага алгоритму Кана полягає в тому, що він працює ітеративно без ризику переповнення стека та автоматично виявляє цикли в графі без додаткових зусиль. Його недоліком є необхідність витратити пам'ять на підрахунок і збереження вхідних ступенів вершин. З іншого боку, алгоритм DFS простіший у реалізації та не вимагає попереднього аналізу ступенів, але має суттєві недоліки у вигляді ризику переповнення стека (StackOverflow) на глибоких графах і потреби в додатковому триколірному фарбуванні вершин для виявлення циклів.

2. Залежність асимптотичної складності від умов: Асимптотична складність обох алгоритмів є однаковою для будь-яких умов, оскільки вони обов'язково обходять кожну вершину та кожне ребро. Часова складність як у найкращому, так і в найгіршому випадках становить $O(|V| + |E|)$. Просторова складність (пам'ять) для обох методів завжди становить $O(|V|)$. При цьому для алгоритму Кана вона витрачається на масив ступенів і чергу, а для DFS — на

масив відвіданих вершин та системний стек викликів, який досягає свого максимуму в найгіршому випадку, коли граф вироджений у лінійний ланцюжок.

3. Застосування до зважених графів: Обидва алгоритми можна повноцінно застосовувати до зважених графів, оскільки ваги на ребрах жодним чином не впливають на сам топологічний порядок, який будується виключно на основі напрямку ребер та зв'язків між вузлами. Порівнюючи їх, алгоритм Кана є зручнішим, якщо паралельно із сортуванням потрібно динамічно обчислювати найкоротші чи найдовші шляхи методом «вперед». DFS частіше використовують для класичного аналізу критичних шляхів у проектах, роблячи розрахунки на кроці повернення з рекурсії («назад»).

4. Вплив структури графа на швидкість: Структура графа безпосередньо впливає на накладні витрати: на розріджених графах з малою кількістю ребер DFS працює дещо швидше, бо не потребує попереднього лінійного проходу для підрахунку ступенів вершин. Тоді як на густих графах з великою кількістю ребер або на графах у вигляді довгих ланцюжків ітеративний алгоритм Кана демонструє кращу стабільність, оскільки DFS у цих умовах здійснює дуже глибокі рекурсивні переходи, що призводить до значних затримок на виділення пам'яті під стек.

5. Обмеження алгоритмів топологічного сортування: Основним обмеженням для обох алгоритмів є наявність циклів у графі, оскільки топологічне сортування можливе лише для ациклічних графів (ОАГ). При цьому алгоритм Кана в такій ситуації просто зупиниться і поверне помилку, а класичний DFS без модифікації зациклиться. Крім того, алгоритм Кана обмежений статичними задачами й найкраще підходить для динамічного керування чергами завдань у реальному часі, тоді як DFS обмежений глибиною стека системи й частіше використовується як складова частина інших комплексних обходів.

6. Шляхи оптимізації алгоритмів: Для оптимізації алгоритму Кана слід використовувати низькорівневі масиви замість важких об'єктів на кшталт set, замінити чергу на швидкий двосторонній список deque та зберігати граф у вигляді компактного списку суміжності для швидкого доступу до пам'яті. В той час як для оптимізації DFS найефективнішим є повна відмова від рекурсії на користь ітеративного підходу з використанням власного явного стека у пам'яті, а також заміна динамічних структур відстеження відвіданих вершин на прості масиви прапорців фіксованого розміру.